SBC Vacuum Controller Runbook

Oct 17, 2026 · @Henry Li

Overview

The vacuum controller runs on a Raspberry Pi 4 Model B (or Pi 5) wired to the RPi5VacuumIO rev A.1 board. It is shipped as one tarball, installed by `install_sbc.py`, and runs as the `sbc-vacuum` systemd service on port 8766. The web UI never talks to the Pi directly: the host's ionbeam-web backend forwards `/api/vacuum/*` to it.

This runbook follows the Operations branch at commit `7072711` (`glasgow_service/deploy/sbc/`).

You need:

- A dev host with the `Development` checkout, Python 3 and SSH access to the Pi.
- A Raspberry Pi 4 Model B, a USB 3 SSD (or a microSD card), and the RPi5VacuumIO board on the 40-way ribbon to J3.
- A user on the Pi with `sudo`.

Safety: nothing switches pumps until you set `"Enable": true` in the profile (step 7). The installer never restarts a running controller, because a restart can power down the pumps.

1. Build the tarball

Run on the dev host, from the `Development/` folder:

```
python3 glasgow_service/deploy/sbc/build_sbc_bundle.py
```

The default output folder is **`glasgow_service/dist/`**, and the file is `sbc-vacuum-<version>.tar.gz`.

- `<version>` = the `version` in `glasgow_service/pyproject.toml` + `+g<8-character commit>`, for example `sbc-vacuum-0.2.0+g70727118.tar.gz`.
- Uncommitted changes under `glasgow_service/` add `.dirty<YYYYMMDDHHMM>`. Commit first for a release build.

Option	Default	Use
`--out <dir>`	`glasgow_service/dist`	Write the tarball somewhere else
`--wheels`	off	Add an offline aarch64 wheelhouse, for a Pi with no internet
`--python <ver>`	`3.11`	Wheel target: `3.11` for Raspberry Pi OS bookworm, `3.13` for trixie

The tarball holds the `glasgow_service` package source (no tests), `install_sbc.py`, the systemd unit and env templates, the board profile template, `wiring-map.json`, `README.md`, `deploy-sbc.sh`, `VERSION` and `SHA256SUMS`.

2. Deploy to the Raspberry Pi SBC

`deploy-sbc.sh` builds the tarball (or takes one with `-b`), copies it to `/tmp/sbc-deploy/` on the Pi over SSH, checks `SHA256SUMS` and runs `sudo python3 install_sbc.py` there. Everything after the host is passed to the installer.

```
# From Development/ on the dev host
glasgow_service/deploy/sbc/deploy-sbc.sh pi@vacuum-pi --dry-run    # show
what it would do
glasgow_service/deploy/sbc/deploy-sbc.sh pi@vacuum-pi              # build +
install

# Ship a tarball you already built
glasgow_service/deploy/sbc/deploy-sbc.sh -b glasgow_service/dist/sbc-vacuum-
<version>.tar.gz pi@vacuum-pi
```

SSH options: `-i <key>` for an identity file, `-p <port>` for a non-standard port. Add `--restart` only when a pump-down is acceptable (see step 7).

No network to the Pi: build with `--wheels`, copy the tarball on a USB stick, then continue at step 6 on the Pi.

3. Install the SBC OS image

On a new Pi, do this before step 2. Use **Raspberry Pi OS Lite (64-bit)**: bookworm (Python 3.11) or trixie (Python 3.13). The installer needs nothing else from the image.

1. Flash the image with Raspberry Pi Imager. Boot media, in order of preference:
   a. USB 3 SSD (e.g. Samsung T7) in a blue USB 3 port.
   b. USB-to-SATA UASP adapter with a 2.5″ SATA SSD. Not a spinning HDD.
   c. microSD only. It works, but wears out under continuous logging. Keep a microSD as the rescue image either way.
2. In Imager's OS customisation: set the hostname (e.g. `vacuum-pi`), a user with a password or SSH key, **enable SSH**, and set the time zone. Use wired Ethernet on the control network.
3. First boot from microSD, update the EEPROM, then set USB boot first: `sudo rpi-eeprom-config --edit` → `BOOT_ORDER=0xf14` (USB, then SD, then retry). Reboot from the SSD.
4. `sudo apt update && sudo apt full-upgrade`, then reboot.
5. With everything unpowered, connect the 40-way ribbon from the Pi header to **J3** on the RPi5VacuumIO board, pin 1 to pin 1. Power the Pi from its own USB-C supply (15 W / 3 A for the Pi 4B).

You don't need to edit `config.txt` by hand: the installer in step 6 adds I2C and the UARTs. Use SSH or HDMI for the console; the serial console pins become the RS-485 bus.

4. Set up the Raspberry Pi

After the first boot from the SSD, set the Pi up as a fixed, locked-down controller on the control network. Replace the values in `< >` with your site's.

1. **Log in** from the dev host: `ssh <user>@vacuum-pi.local` (or the address your DHCP server gave it).
2. **SSH keys.** Copy your key, check you can log in with it, then turn off password logins:

```
ssh-copy-id <user>@vacuum-pi.local
echo 'PasswordAuthentication no' | sudo tee /etc/ssh/sshd_config.d/10-keys-only.conf
sudo systemctl reload ssh
```

3. **Static IP.** The host backend reaches the Pi by address (`VACUUM_CONTROLLER_URL`), so it must not change. Raspberry Pi OS bookworm uses NetworkManager:

```
nmcli con show                                    # find the wired
connection name
sudo nmcli con mod "Wired connection 1" ipv4.method manual \
  ipv4.addresses <pi-ip>/24 ipv4.gateway <gateway> ipv4.dns <dns>
sudo nmcli con up "Wired connection 1"
```

4. **Time.** Set the time zone and check that the clock synchronises, so logs and alerts line up with the host: `sudo timedatectl set-timezone <Area/City>`, then `timedatectl` must show `System clock synchronized: yes`.
5. **Firmware.** `sudo rpi-eeprom-update -a`, reboot, and check that `BOOT_ORDER` is still `0xf14` (`rpi-eeprom-config`).
6. **Firewall.** Allow SSH, and port 8766 from the host only:

```
sudo apt install ufw
sudo ufw allow OpenSSH
sudo ufw allow from <host-ip> to any port 8766 proto tcp
sudo ufw enable
```

7. **Updates.** Run `sudo apt update && sudo apt full-upgrade` only in a maintenance window. Don't enable automatic reboots: a reboot drops relay K1, which can power down every pump.
8. **Check:** `hostnamectl`, `ip -4 addr`, and `ping <host-ip>`.

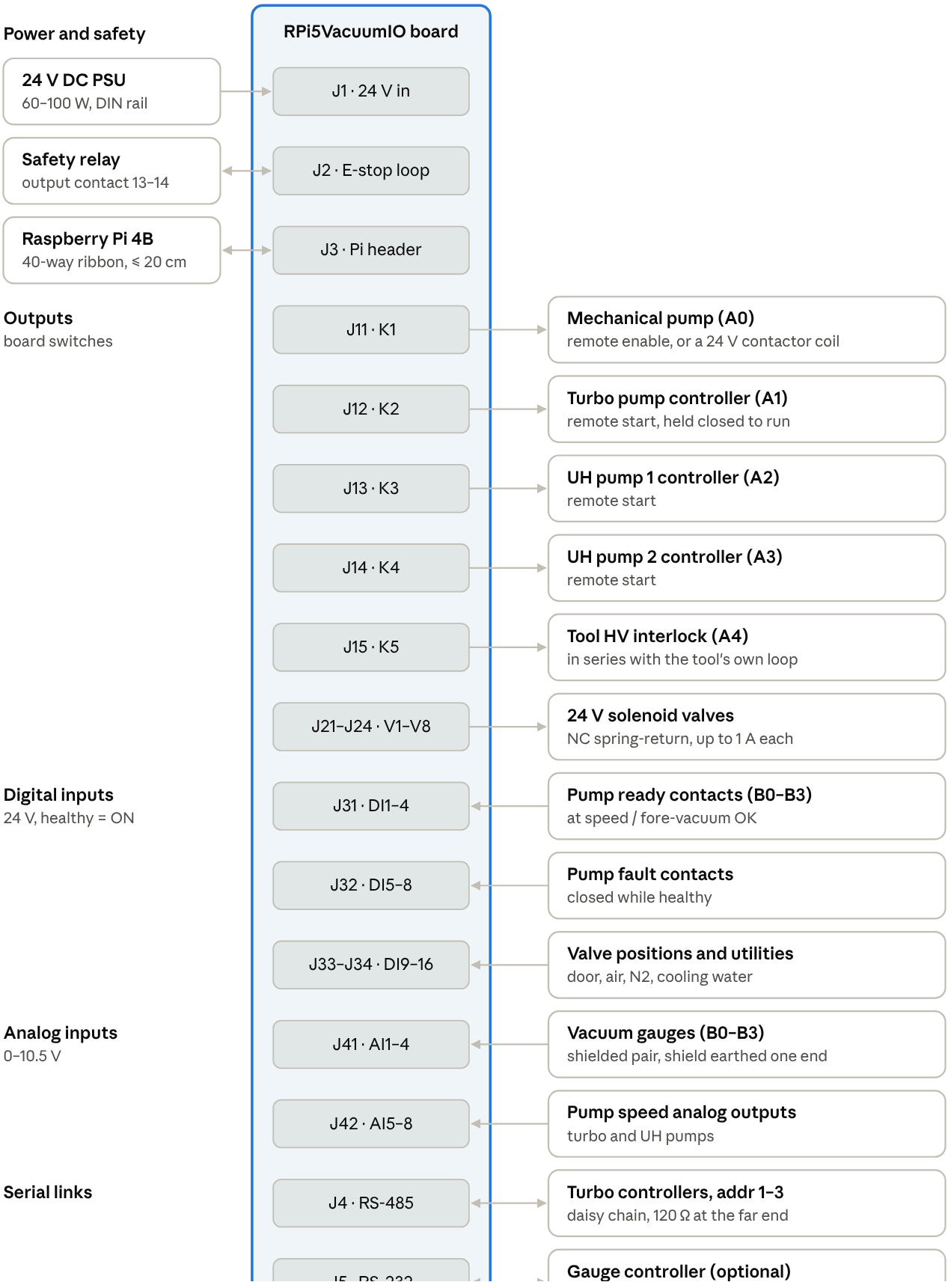
Don't turn on I2C or the UARTs in `raspi-config`: the installer in step 6 writes them to `config.txt`.

5. Wire the Pi to the I/O board and the tool

Every connection to the tool goes through a connector on the RPi5VacuumIO board; the Pi itself only connects to J3. Wire with the 24 V supply and the tool's pump controllers off and locked out, and keep `"Enable": false` until commissioning (step 7).

The Pi reaches the tool only through the I/O board connectors

RPi5VacuumIO rev A.1 · arrows show signal direction





→ the board drives it ← it reports to the board ↔ both ways

RPi5VacuumIO wiring map · 16 connectors

Wire in this order, checking each LED before the next step:

1. **J1, 24 V in:** PSU +V to pin 1, 0 V to pin 2. Bond the PSU 0 V to protective earth at one point. Green **D3** comes on.
2. **J2, E-stop loop:** a safety-relay output contact between J2-1 and J2-2 (a second contact goes into the tool's own pump / HV enable chain). Yellow **D9** comes on when the loop is closed.
3. **J3, Pi:** 40-way ribbon, pin 1 to pin 1 (red stripe, square pad). The Pi keeps its own USB-C supply; J3 does not power it.
4. **J11–J15, relays:** COM to pin 1, NO to pin 2, into each controller's *remote enable / start* input. Never pump mains power: a plain mains pump needs a rated 24 V contactor.
5. **J31–J34, inputs:** dry contacts between `+24` and `DIn`, COM to 0 V. Wire fault inputs so they are ON while healthy.
6. **J21–J24, valves:** each coil between `+24` and `Vn−`; normally-closed, spring-return valves.
7. **J41–J42, gauges:** one shielded twisted pair per gauge, shield earthed at the board end only.
8. **J4, RS-485:** daisy chain to the three turbo controllers (addresses 1–3), JP9 bridged on the board, 120 Ω at the last controller.

After the installer and its reboot (step 6), `i2cdetect -y 1` must show **20, 21, 48, 49**. Full pin tables: `GlasgowDataIO/Hardware/VacuumController/RPi5VacuumIO/WIRING.md`.

6. Install on the SBC

`deploy-sbc.sh` runs this for you. To install by hand (USB stick, or a tarball already on the Pi):

```
tar xzf sbc-vacuum-<version>.tar.gz
cd sbc-vacuum-<version>/
sha256sum -c SHA256SUMS
sudo python3 install_sbc.py --dry-run     # show the actions, change nothing
sudo python3 install_sbc.py
```

The installer:

1. Detects the Pi model from `/proc/device-tree/model` (`--model 4B|5` to confirm).
2. Installs `python3-venv`, `python3-pip`, `python3-lgpio`, `i2c-tools` (`--no-apt` skips this).
3. Creates the service user `vacuum` in `gpio`, `i2c` and `dialout`.
4. Adds a managed block to `config.txt` (backup `config.txt.sbc-vacuum.bak-<time>`). Pi 4B: `dtparam=i2c_arm=on`, `enable_uart=1`, `dtoverlay=disable-bt`, `dtoverlay=uart5`. It removes the serial console and disables `hciuart`. Any of these means **reboot required**.
5. Installs the release to `/opt/ionbeam/sbc-vacuum/releases/<version>/` with its own virtualenv, keeping the last 3 releases.
6. Writes site files once, never overwriting them (a new copy lands next to them as `.dist`):

File	Holds
`/etc/sbc-vacuum/vacuumSystem.json`	The profile for this model, with `"Enable": false` and `"IsProduction": true`
`/etc/sbc-vacuum/wiring-map.json`	The as-wired connection map; edit it if the cabinet differs
`/etc/sbc-vacuum.env`	Port 8766, the profile path and a random `SBC_VACUUM_TOKEN`
`/etc/sbc-vacuum/WIRING-MAP.md`	The map as tables

7. Checks the profile against the wiring map. If it fails, the release is not activated (`--force` overrides).
8. Points `/opt/ionbeam/sbc-vacuum/current` at the new release and enables the `sbc-vacuum` unit. It does **not** restart a running controller unless you pass `--restart`.

```
sudo python3 install_sbc.py --rollback          # swap to the previous
release (no restart)
sudo python3 install_sbc.py --uninstall --yes   # stops the controller; keeps
/etc and /opt
```

7. Verify and start the SBC

1. Reboot if the installer said so: `sudo reboot`.
2. Check the Pi, `config.txt`, the console, the `/dev` nodes, the groups and the profile against the map:

   ```
   sudo sbc-vacuum-preflight preflight
   ```

3. Probe the board with the service stopped (pumps go off). It reads only, and must find 0x20, 0x21, 0x48, 0x49 and the rails:

   ```
   sudo systemctl stop sbc-vacuum
   sudo sbc-vacuum-preflight preflight --probe
   ```

4. Commission with WIRING.md step 12 (jumper test; `"Interlock": false` on the gauges only for that test).
5. Turn control on: set `"Enable": true` in `/etc/sbc-vacuum/vacuumSystem.json`, then start it:

   ```
   sudo systemctl restart sbc-vacuum
   systemctl status sbc-vacuum
   journalctl -u sbc-vacuum -f
   ```

6. Check the API on the Pi:

Check	Expect
`curl http://127.0.0.1:8766/health/ready`	200: the process is ready
`curl http://127.0.0.1:8766/status`	`"vacuum_enabled": true`, `"mode": "board"`
`curl http://127.0.0.1:8766/vacuum`	Pump list with the cascade advancing from `MechanicalVacuumPump`
`curl -i http://127.0.0.1:8766/health/vacuum`	200 once running; 503 with `problems` while latched, alarmed or stale

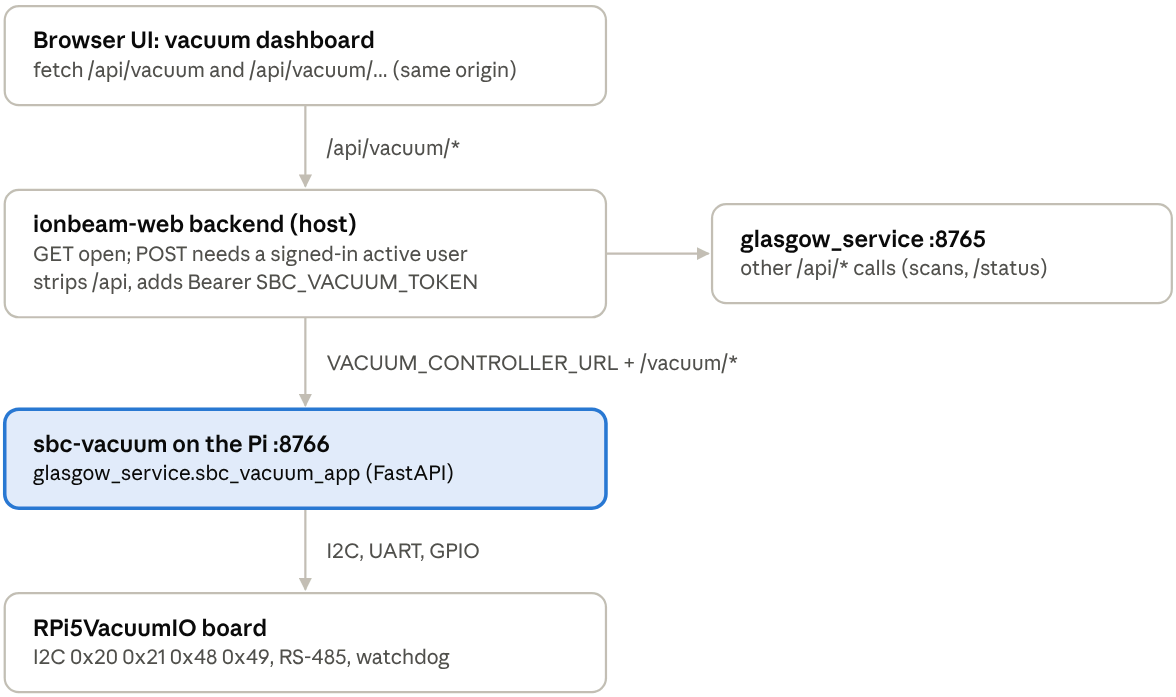
7. Point monitoring at `http://<pi>:8766/health/vacuum` and alert on any non-200. Optionally set `SBC_VACUUM_ALERT_WEBHOOK` in `/etc/sbc-vacuum.env`.

The unit runs the wiring check before every start, and systemd restarts it if its control loop hangs (`WatchdogSec=20`). A restart can power down every pump, including the backing pump unless `KeepBackingPumpOnRestart` is verified on this board.

8. How the UI talks to the SBC

The browser calls `/api/vacuum/*` on the host's ionbeam-web backend. The backend strips `/api`, adds `Authorization: Bearer <SBC_VACUUM_TOKEN>` and forwards to `VACUUM_CONTROLLER_URL`. Scan traffic goes to glasgow_service and never touches the vacuum GPIO.

The UI reaches the SBC only through the host backend



vacuum control path · browser to board

On the **host**, set these in the web backend's environment (the systemd unit, or its secrets file), then restart the backend:

```
VACUUM_CONTROLLER_URL=http://<pi>:8766        # default http://127.0.0.1:8766
SBC_VACUUM_TOKEN=<the token from /etc/sbc-vacuum.env on the Pi>
```

The backend also needs Configuration → General → **Active vacuum control** on. If that's off, or the SBC reports itself disabled, `/api/vacuum/*` returns 404 and the dashboard is hidden.

UI / backend path	SBC path	Method	Used for
`/api/vacuum`	`/vacuum`	GET	Dashboard state: pumps, readings, valves, alarms, `cascade_latch` (polled)
`/api/vacuum/pumps/{name}/power`	`/vacuum/pumps/{name}/power`	POST	Pump on/off toggle on a pump card
`/api/vacuum/resume`	`/vacuum/resume`	POST	Resume after an operator or safety stop
`/api/vacuum/high-voltage/power`	`/vacuum/high-voltage/power`	POST	High voltage on/off (vacuum-ready interlocked)
`/api/vacuum/stop`	`/vacuum/stop`	POST	Stop the cascade (backing pump stays on)
`/api/vacuum/pumps/MechanicalVacuumPump/restart`	same, without `/api`	POST	Power-cycle the backing pump; optional `{"off_seconds": 3}`
`/api/vacuum/acquire`, `/api/vacuum/release`	`/vacuum/acquire`, `/vacuum/release`	POST	Take or release control
— (backend only)	`/status`	GET	Backend checks `vacuum_enabled` to decide whether to show vacuum
— (monitoring)	`/health/live`, `/health/ready`, `/health/vacuum`	GET	Liveness, process ready, continuous vacuum health (503 when latched)

The UI calls the first four; the rest exist on the SBC and pass through the same proxy. Every POST on the SBC needs the bearer token; GETs don't. `/vacuum/simulation/*` and `/emulator/*` exist only when the emulator is active (`"IsProduction": false`).